

Codage de Huffman

Le codage de Huffman est une technique permettant de compresser un texte : l'idée de base est de coder les caractères avec un code de longueur variable. Pour coder un texte, on commence par calculer les fréquences des différents caractères du texte, la fréquence d'un caractère étant égale à son nombre d'occurrences divisé par le nombre total de caractères du texte. On associe ensuite un code binaire à chaque caractère de telle sorte que les caractères les plus fréquents soient associés à des codes binaires courts (un ou quelques bits) alors que les caractères les moins fréquents sont à l'inverse associés aux codes binaires les plus longs.

Nous nous proposons dans cet exercice de travailler sur le principe de cette technique en adoptant les simplifications suivantes :

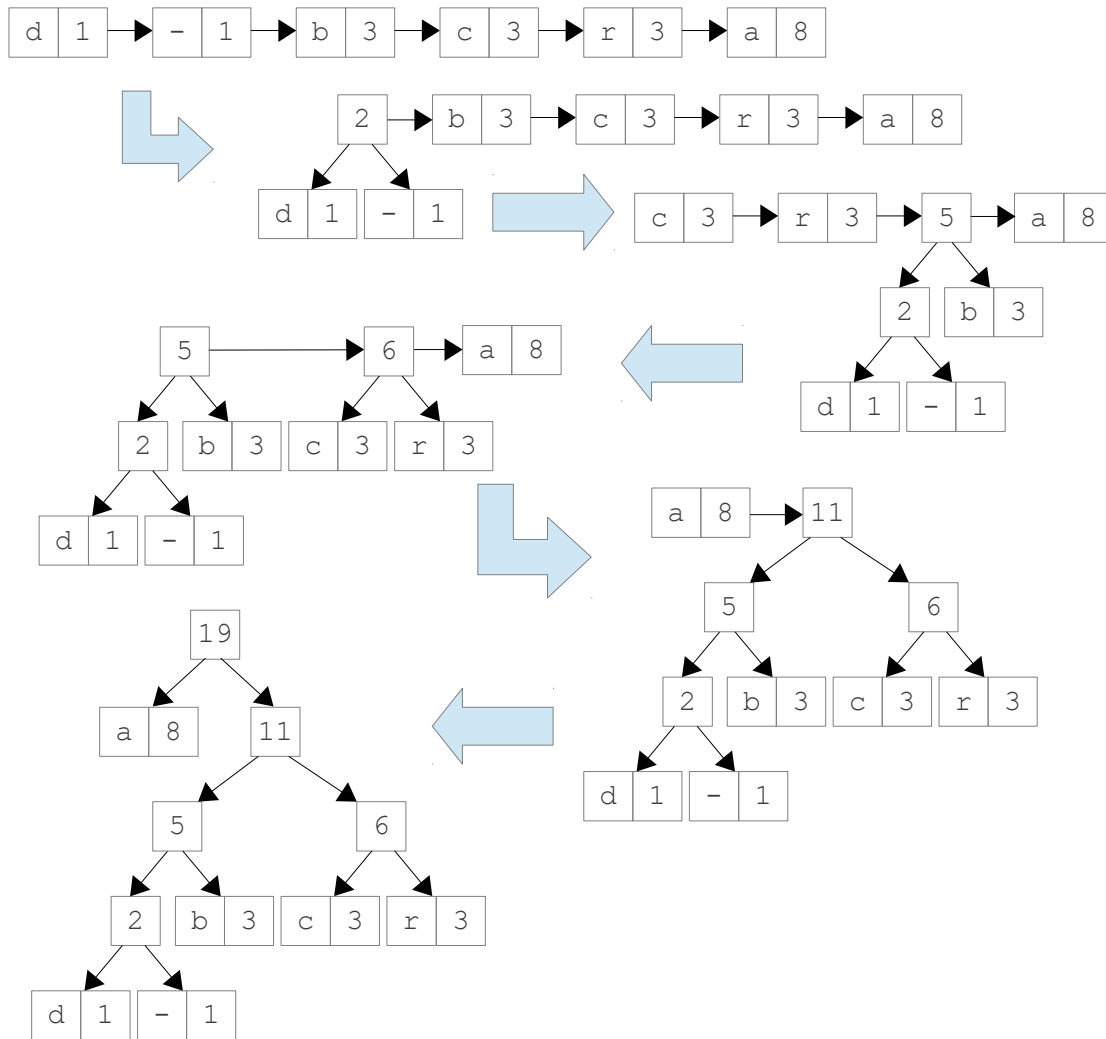
- les fréquences étant par définition proportionnelles aux nombres d'occurrences de chaque caractère, nous travaillons à partir des nombres d'occurrences plutôt que des fréquences ;
- l'effet de compression est normalement obtenu en ne représentant plus systématiquement chaque caractère sur 8 ou 16 bits (selon le jeu de caractères choisi) mais sur un nombre variables de bits, nous nous contentons ici de simuler la représentation codée des textes par des chaînes de caractères contenant uniquement les caractères '0' ou '1' plutôt que de travailler directement sur des bits.

Le principe de ce codage repose sur la construction d'un arbre binaire (appelé arbre de Huffman) dans lequel chaque nœud non feuille possède exactement 2 nœuds fils et chaque feuille correspond à l'un des caractères du texte à compresser. En outre, chaque nœud de l'arbre de Huffman (qu'il soit feuille ou non feuille) est pondéré par le nombre d'occurrences total des caractères associés au sous-arbre issu de ce nœud. Une fois le décompte des occurrences de chaque caractère effectué, on peut construire cet arbre de la manière suivante, grâce à une liste.

- (1) La liste contient initialement un nœud feuille pour chaque caractère. Ces feuilles sont triées dans la liste dans l'ordre croissant du nombre d'occurrences de chaque caractère.
- (2) On retire les deux éléments de la liste portant les nombres d'occurrences les plus faibles.
- (3) On construit un nouveau nœud de l'arbre ayant pour fils les éléments retirés de la liste en (2), et on associe à ce nouveau nœud un nombre d'occurrences correspondant à la somme des nombres d'occurrences de ces deux éléments.
- (4) On place ce nouveau nœud dans la liste, de telle sorte que celle-ci reste triée.
- (5) On reprend en (2) tant que la liste possède au moins 2 éléments. Lorsque celle-ci ne contient plus qu'un élément, cet élément unique est la racine de l'arbre de Huffman.

Par exemple, sur le texte "abracadabra-baccara", les étapes possibles de la construction de l'arbre de Huffman sont montrées page suivante.

L'arbre de Huffman détermine le code binaire qui est ensuite associé à chaque caractère. Ce code correspond au parcours effectué depuis la racine de l'arbre pour atteindre le caractère en question : à chaque fois que l'on suit un lien vers le fils gauche on note 0 par convention et 1 si l'on suit le lien vers le fils droit. On peut alors construire un dictionnaire associant un code binaire à chaque caractère pour coder des textes, l'arbre étant surtout utile dans le décodage.



Sur l'exemple ci-dessus, on obtient le dictionnaire

{('a':0), ('b':101), ('c':110), ('d':1000), ('r':111), ('-':1001)}

qui permet de coder "abracadabra-baccara" en

"0101111011001000010111101001101011011001110".

Nous considérons dans cet exercice les 7 classes ci-dessous dont la définition n'est que partiellement donnée.

```
package huffman;
import java.util.Map;

public class Huffman {
    private Map<Character, String> dictionnaire;
    private NoeudAbstrait arbre;

    public Huffman(String texte) {
        initArbre(compteCaracteres(texte));
        initDictionnaire();
    }

    private Map<Character, Integer> compteCaracteres(String texte) {...}
    private void initArbre(Map<Character, Integer> comptes) {...}
    private void initDictionnaire() {...}
    public NoeudAbstrait getArbre() {...}
    public Map<Character, String> getDictionnaire() {...}
    public String code(String texte) throws CaractereInconnuException {...}
    public String decode(String texte) throws FinDeTexteInattendueException {...}
}
```

```

package huffman;
import java.util.Map;

public class NoeudAbstrait implements Comparable<NoeudAbstrait> {
    private int poids;

    public NoeudAbstrait(int poids) {...}
    public NoeudAbstrait() {...}

    public int getPoids() {...}
    public int compareTo(NoeudAbstrait n) {...}
    public abstract void fournirCodes(Map<Character, String> m, String prefixe);
    public abstract Character getNextChar(String s)
                                                throws FinDeTexteInattendueException;
    public abstract int hauteur();
}

```

```

package huffman;
import java.util.Map;

public class Noeud extends NoeudAbstrait {
    private NoeudAbstrait gauche;
    private NoeudAbstrait droit;

    public Noeud(int poids, NoeudAbstrait gauche, NoeudAbstrait droit) {...}

    public void fournirCodes(Map<Character, String> m, String prefixe) {...}
    public Character getNextChar(String s) throws FinDeTexteInattendueException {...}
    public int hauteur() {...}
}

```

```

package huffman;
import java.util.Map;

public class Feuille extends NoeudAbstrait {
    private Character caractere;

    public Feuille(Character c, int poids) {...}

    public void fournirCodes(Map<Character, String> m, String prefixe) {...}
    public Character getNextChar(String s) throws FinDeTexteInattendueException {...}
    public int hauteur() {...}
}

```

```

package huffman;

public class CaractereInconnuException extends Exception {...}

```

```

package huffman;

public class FinDeTexteInattendueException extends Exception {...}

```

```

package huffman;
import java.util.Collection;
import java.util.LinkedList;

public class ListeTrie extends LinkedList<NoeudAbstrait> {
    public ListeTrie() {...}
    public ListeTrie(Collection<? extends NoeudAbstrait> c) {...}

    public boolean add(NoeudAbstrait n) {...}
    public boolean addAll(Collection<? extends NoeudAbstrait> c) {...}
}

```

1. Structure de l'arbre de Huffman

Les classes `NoeudAbstrait`, `Noeud` et `Feuille` fournissent la structure de l'arbre de Huffman que nous désirons construire lors d'un codage.

Définir les constructeurs de ces classes.

Définir les méthodes `getPoids` et `compareTo` de la classe `NoeudAbstrait`, sachant que comparer des `NoeudAbstrait` revient à comparer leur poids respectifs.

2. Classe `ListeTrie`

La classe `ListeTrie` est destinée à être utilisée lors de la construction de l'arbre de Huffman. Au delà des services habituellement rendus par une `LinkedList`, une `ListeTrie` doit maintenir ses éléments dans un ordre croissant, en s'appuyant sur le `compareTo` défini dans la classe `NoeudAbstrait`. En supposant que les éléments d'une `ListeTrie` sont immutables, la redéfinition de la méthode `add` suffit à maintenir cet ordre.

Définir les constructeurs et les méthodes `add` et `addAll` de la classe `ListeTrie`.

3. Construction d'un codage de Huffman

La construction d'un codage de Huffman est réalisée à l'aide d'un texte de référence. Cette construction consiste en la construction d'un arbre de Huffman en premier lieu, puis d'un dictionnaire.

3.1 Décompte des caractères du texte de référence

Définir la méthode `compteCaracteres` de la classe `Huffman`. Cette méthode renvoie une `Map` qui associe à chaque caractère le nombre de ses occurrences dans le texte fourni en paramètre. Seuls les caractères présents dans le texte fourni en paramètre sont présents dans cette `Map`.

3.2 Construction de l'arbre de Huffman

Définir la méthode `initArbre` de la classe `Huffman`. Cette méthode construit l'arbre de Huffman et initialise la variable d'instance `arbre`.

3.3 Construction du dictionnaire de codage

Définir les méthodes `fournitCodes` des classes de la hiérarchie d'héritage issue de la classe `NoeudAbstrait`. `fournitCodes` invoquée sur un `NoeudAbstrait` a pour effet d'ajouter dans la `Map` fournie en paramètre les associations (caractère – code binaire) correspondant aux caractères présents dans les feuilles du sous-arbre dont il est racine, chaque code binaire étant préfixé par la `String` fournie en paramètre.

Définir la méthode `initDictionnaire` de la classe `Huffman`. Cette méthode initialise la variable d'instance `dictionnaire` qui associe chaque caractère du texte de référence à un code binaire donné sous la forme d'une `String`.

4. Codage d'un texte

C'est essentiellement le dictionnaire qui est utilisé lors du codage d'un texte par un codage de Huffman. Si le texte à coder contient un caractère non présent dans le dictionnaire de codage, le codage s'avère impossible.

Définir la méthode `code` de la classe `Huffman` qui retourne sous la forme d'une `String` le codage binaire de la `String` fournie en paramètre. Une exception `CaractereInconnuException` est levée si un caractère absent du codage de Huffman est rencontré.

5. Décodage d'un texte

Décoder un texte grâce à un codage de Huffman repose essentiellement sur l'utilisation de l'arbre de Huffman correspondant. Il s'agit de suivre les liens gauche ou droit depuis la racine de l'arbre suivant que des '0' ou des '1' sont lus dans le texte codé jusqu'à trouver un caractère, puis continuer en recommençant un parcours depuis la racine. Si la lecture de la fin du texte ne permet pas de terminer sur une feuille de l'arbre, le décodage s'avère impossible. On suppose dans cet exercice que le texte à décoder ne contient que des caractères '0' ou '1'.

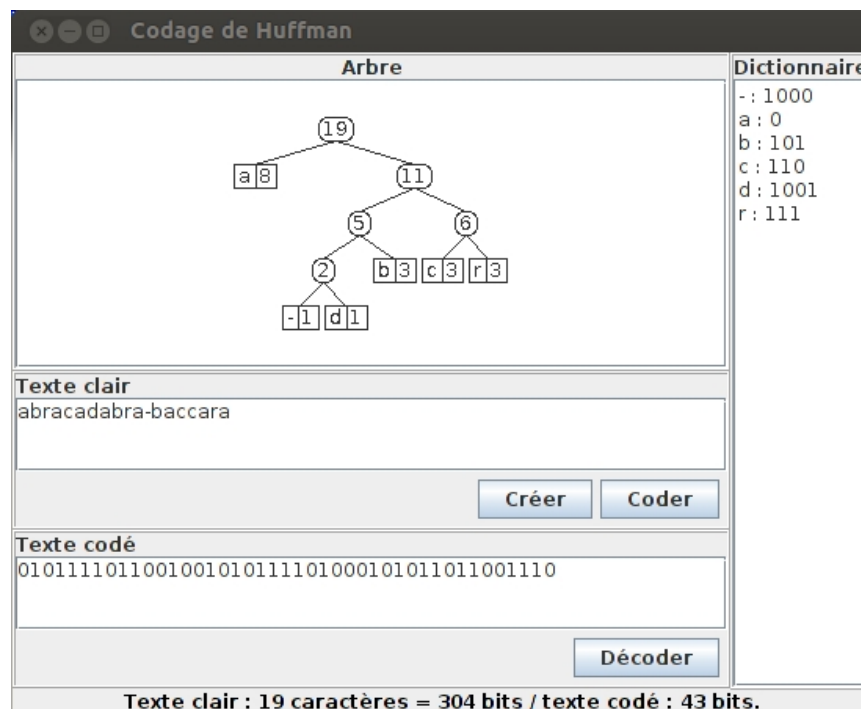
Définir les méthodes `getNextChar` de la hiérarchie d'héritage issue de la classe `NoeudAbstrait`. `getNextChar` invoquée sur un `NoeudAbstrait` retourne le caractère correspondant à la feuille atteinte depuis ce `NoeudAbstrait` en se guidant grâce aux premiers caractères de la `String` fournie en paramètre. Si ce parcours dans l'arbre n'aboutit pas à une Feuille, une exception `FinDeTexteInattendueException` est levée.

Définir la méthode `decode` de la classe `Huffman` qui retourne une `String` correspondant au décodage d'un codage binaire donné en paramètre sous la forme d'une `String`.

6. Interface graphique

6.1 Description

Réaliser une application graphique similaire à celle-ci :



- L'appui sur le bouton **Créer** permet de créer un codage de Huffman en prenant le texte présent dans la zone **Texte clair** comme texte de référence. À la création de ce codage son arbre s'affiche dans la zone **Arbre** et son dictionnaire dans la zone **Dictionnaire**.
- L'appui sur le bouton **Coder** permet de coder le contenu de la zone **Texte clair**, le résultat est affiché dans la zone **Texte codé** et un texte apparaît en bas de la fenêtre pour indiquer le nombre de bits nécessaire à la représentation de chaque texte.
- L'appui sur le bouton **Décoder** permet de décoder le contenu de la zone **Texte codé**, le résultat est affiché dans la zone **Texte clair** et un texte apparaît en bas de la fenêtre pour indiquer le nombre de bits nécessaire à la représentation de chaque texte.
- S'il n'est pas possible de coder ou décoder, un texte précisant l'anomalie apparaît en bas de la fenêtre (à la place où s'affiche normalement le nombre de bits nécessaire à chaque représentation).

6.2 Indications techniques

Les exceptions `CaractereInconnuException` et `FinDeTexteInattendueException` doivent être récupérées dans votre application.

Pour le dessin de l'arbre, il est possible d'utiliser la classe `DessinHuffman`. Pour cela il suffit lors de la conception de votre application graphique d'utiliser un `JPanel` pour la zone **Arbre**. Dans le code source de votre application, il faut ensuite changer le type de l'attribut référençant ce `JPanel` en `DessinHuffman`. La classe `DessinHuffman` définit une méthode **public void** `setCodage(Huffman codage)` qui permet d'associer un codage de Huffman au composant graphique.

Pour que la classe `DessinHuffman` fonctionne correctement, il est nécessaire d'ajouter les accesseurs en lecture suivants :

- **public** `Character` `getCaractere()` dans la classe `Feuille`,
- **public** `NoeudAbstrait` `getGauche()` et **public** `NoeudAbstrait` `getDroit()` dans la classe `Noeud`.

La méthode **public** `int` `hauteur()` doit également être correctement définie dans les classes de la hiérarchie des nœuds.